# Homework 2 - ECG Arrhythmia Classification

During this homework, I learned to integrate biomedical signal processing, data analysis, and machine learning to classify ECG signals into normal and arrhythmic segments.

In [ ]:

In [12]:
```
!pip install -q wfdb numpy pandas matplotlib scipy
```

```
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

# Step 1: Data Acquisition and Preprocessing

In [10]:
```
import wfdb
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt
import random

def bandpass_filter(signal, lowcut=0.5, highcut=40, fs=360, order=4):
    nyq = 0.5 * fs  # Nyquist frequency
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    y = filtfilt(b, a, signal)
    return y

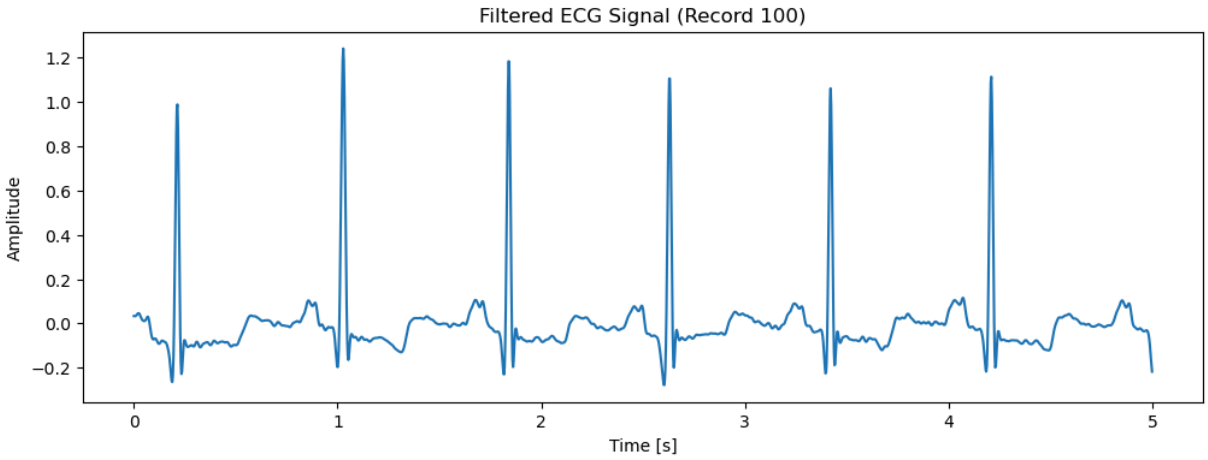
# Load ECG signal
record = wfdb.rdrecord('100', pn_dir='mitdb', channels=[0])
annotation = wfdb.rdann('100', 'atr', pn_dir='mitdb')

# Extract signal and sampling rate
ecg_signal = record.p_signal.flatten()
fs = record.fs

# Filter the ECG signal
ecg_signal_filtered = bandpass_filter(ecg_signal, fs=fs)

# Plot the first 5 seconds
plt.figure(figsize=(12,4))
plt.plot(np.arange(len(ecg_signal[:5*fs]))/fs, ecg_signal_filtered[:5*fs])
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
```

```
plt.title("Filtered ECG Signal (Record 100)")
plt.show()
```




In [14]:
```
import wfdb
import numpy as np
import matplotlib.pyplot as plt

# Example: Record 100 from MIT-BIH
record = wfdb.rdrecord('100', pn_dir='mitdb')  # ECG signal
annotation = wfdb.rdann('100', 'atr', pn_dir='mitdb')  # annotations

# Extract ECG and sampling frequency
ecg_signal = record.p_signal[:,0]  # first lead
fs = record.fs

# Extract annotation samples and symbols
ann_samples = annotation.sample       # indices of annotated beats
ann_symbols = annotation.symbol       # beat type symbols
```

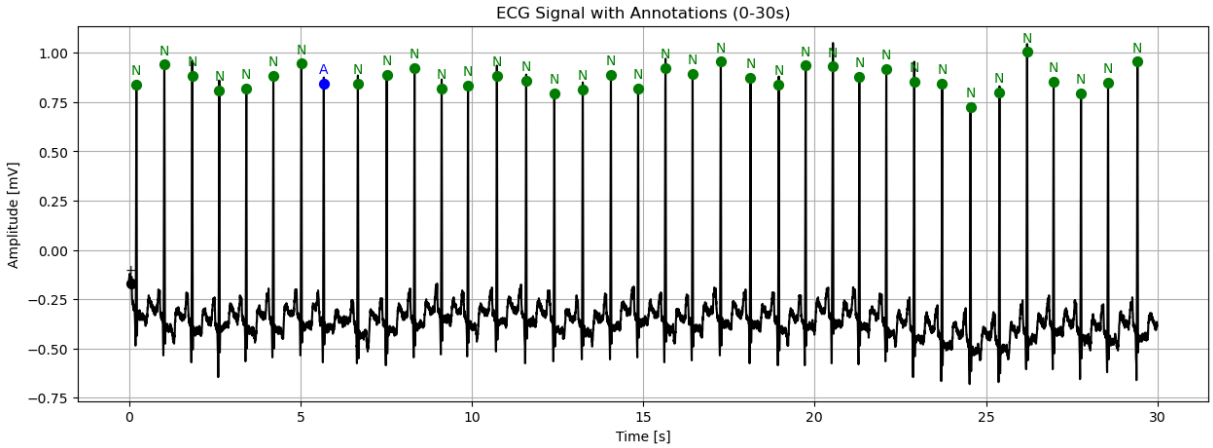
In [15]:
```
start_sec = 0
end_sec = 30
start_sample = int(start_sec * fs)
end_sample = int(end_sec * fs)

beat_colors = {'N': 'green', 'V': 'red', 'A': 'blue', 'L': 'purple', 'R': 'c

plt.figure(figsize=(15,5))
plt.plot(np.arange(start_sample, end_sample)/fs, ecg_signal[start_sample:enc

for sample, symbol in zip(ann_samples, ann_symbols):
    if start_sample <= sample <= end_sample:
        color = beat_colors.get(symbol, 'black')
        plt.scatter(sample/fs, ecg_signal[sample], color=color, s=50, zorder
        plt.text(sample/fs, ecg_signal[sample]+0.05, symbol, color=color, fc

plt.xlabel("Time [s]")
plt.ylabel("Amplitude [mV]")
plt.title(f"ECG Signal with Annotations ({start_sec}-{end_sec}s)")
plt.grid(True)
plt.show()
```




In [17]:
```
# MIT-BIH records (common ones)
mitbih_records = ['100','101','102','103','104','105','106','107','108','109
                  '111','112','113','114','115','116','117','118','119',
                  '121','122','123','124','200','201','202','203','205','207
                  '208','209','210','212','213','214','215','217','219','220
                  '221','222','223','228','230','231','232','233','234']

# Randomly pick 10 subjects
random.seed(42)  # for reproducibility
selected_records = random.sample(mitbih_records, 10)
print("Selected subjects:", selected_records)
```

```
Selected subjects: ['222', '107', '101', '118', '116', '115', '108', '106',
'214', '105']
```

In [18]:
```
all_beats = []  # to store beat segments and labels

for record_id in selected_records:
    # Load ECG and annotations
    record = wfdb.rdrecord(record_id, pn_dir='mitdb', channels=[0])
    annotation = wfdb.rdann(record_id, 'atr', pn_dir='mitdb')

    ecg_signal = record.p_signal.flatten()
    fs = record.fs

    # Filter ECG
    ecg_filtered = bandpass_filter(ecg_signal, fs=fs)

    # Segment around R-peaks (e.g., 0.2s before and 0.4s after)
    window_before = int(0.2 * fs)
    window_after = int(0.4 * fs)

    for sample, symbol in zip(annotation.sample, annotation.symbol):
        start = max(sample - window_before, 0)
        end = min(sample + window_after, len(ecg_filtered))
        beat = ecg_filtered[start:end]

        # Pad if necessary
        if len(beat) < (window_before + window_after):
            beat = np.pad(beat, (0, window_before + window_after - len(beat)

        # Label: Normal (N) vs Arrhythmia (all other symbols)
```

```
        label = 'Normal' if symbol == 'N' else 'Arrhythmia'

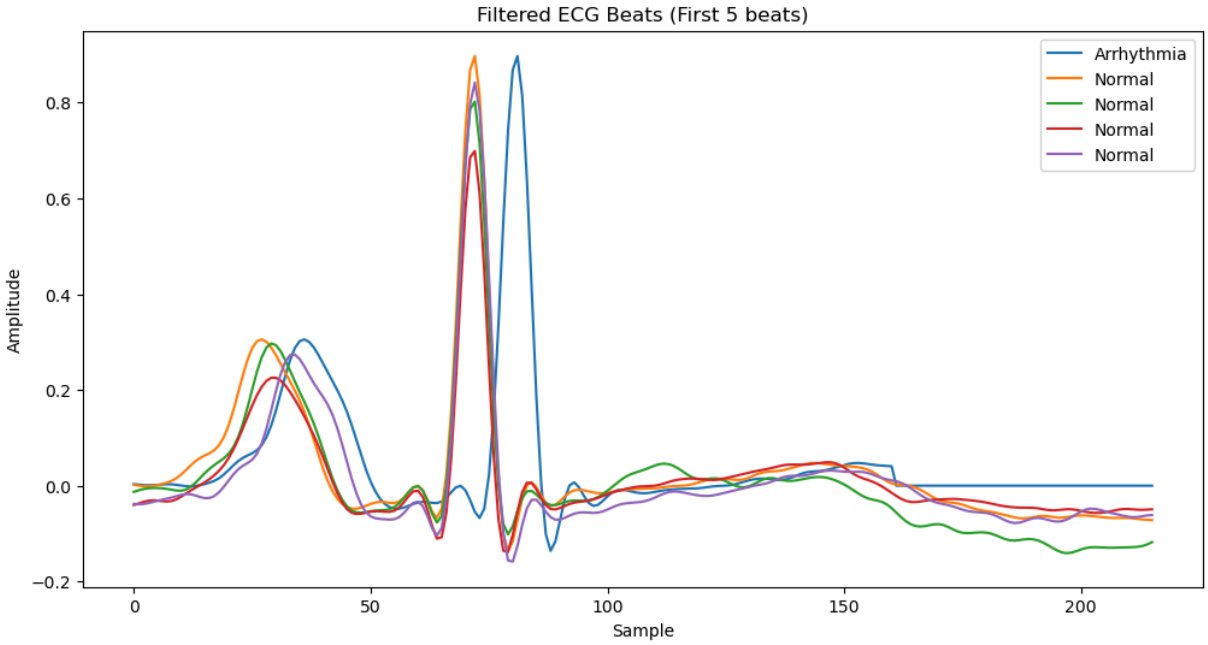
        # Save beat and label
        all_beats.append({'record': record_id, 'r_peak': sample, 'beat': bea

# Convert to DataFrame
df_beats = pd.DataFrame(all_beats)
print(df_beats.head())
```

```
  record  r_peak                                                beat  \
0    222      17  [0.003211971516413185, 0.0020895512110316967, ...
1    222      81  [0.0017242292867025244, 0.00017881999468177562...
2    222     361  [-0.012939478634611116, -0.009694406201424916,...
3    222     646  [-0.04038034915317255, -0.037025382583211745, ...
4    222     892  [-0.03812538877246667, -0.038735806941911415, ...

        label
0  Arrhythmia
1      Normal
2      Normal
3      Normal
4      Normal
```

In [19]:
```
plt.figure(figsize=(12,6))
for i in range(5):
    plt.plot(df_beats['beat'][i], label=df_beats['label'][i])
plt.xlabel("Sample")
plt.ylabel("Amplitude")
plt.title("Filtered ECG Beats (First 5 beats)")
plt.legend()
plt.show()
```




In [1]:
```
# STEP 1: build windows dataset from MIT-BIH
import wfdb
import numpy as np
import pandas as pd
```

```python
from scipy.signal import butter, filtfilt, welch
import os

SAMPLE_RATE = 360       # MIT-BIH sampling freq
WINDOW_SECONDS = 10     # set 10..30 as required
WINDOW_SAMPLES = WINDOW_SECONDS * SAMPLE_RATE
STEP_SECONDS = WINDOW_SECONDS  # non-overlapping; set <WINDOW_SECONDS to ove
MARGIN_SECONDS_AROUND_ANN = 0.05  # mark +/-50 ms around abnormal annot
CHANNEL = 0             # which lead to use (0 or 1)
OUTPUT_CSV = 'mitbih_windows.csv'

# full list of MIT-BIH record ids (48 records)
RECORDS = [
 '100','101','102','103','104','105','106','107','108','109',
 '111','112','113','114','115','116','117','118','119','121',
 '122','123','124','200','201','202','203','205','207','208',
 '209','210','212','213','214','215','217','219','220','221',
 '222','223','228','230','231','232','233','234'
]

def bandpass_filter(x, fs=SAMPLE_RATE, low=0.5, high=40, order=3):
    nyq = 0.5 * fs
    lowcut = low / nyq
    highcut = high / nyq
    b, a = butter(order, [lowcut, highcut], btype='band')
    return filtfilt(b, a, x)

def extract_features_segment(sig, fs=SAMPLE_RATE):
    x = sig.astype(np.float64)
    feats = {}
    feats['mean'] = float(np.mean(x))
    feats['std'] = float(np.std(x))
    feats['min'] = float(np.min(x))
    feats['max'] = float(np.max(x))
    feats['median'] = float(np.median(x))
    # use pandas Series for skew/kurtosis
    import pandas as pd
    s = pd.Series(x)
    feats['skew'] = float(s.skew())
    feats['kurtosis'] = float(s.kurtosis())
    feats['ptp'] = float(np.ptp(x))
    # Welch spectral features
    f, Pxx = welch(x, fs=fs, nperseg=min(len(x), fs*2))
    def band_power(lo, hi):
        mask = (f >= lo) & (f <= hi)
        if mask.sum() == 0:
            return 0.0
        return float(np.trapz(Pxx[mask], f[mask]))
    feats['power_0_5'] = band_power(0.5, 5)
    feats['power_5_15'] = band_power(5, 15)
    feats['power_15_40'] = band_power(15, 40)
    feats['total_power'] = float(np.trapz(Pxx, f))
    Pnorm = Pxx / (Pxx.sum() + 1e-12)
    feats['spec_entropy'] = float(-np.sum(Pnorm * np.log(Pnorm + 1e-12)))
    return feats
```

```python
def build_windows_for_record(record_name, channel=CHANNEL):
    rec = wfdb.rdrecord(record_name, pn_dir='mitdb')
    ann = wfdb.rdann(record_name, 'atr', pn_dir='mitdb')
    sig = rec.p_signal[:, channel]
    filt = bandpass_filter(sig, fs=rec.fs)
    n_samples = filt.shape[0]

    # mark samples as arrhythmic if annotation symbol != 'N'
    arr_mask = np.zeros(n_samples, dtype=bool)
    margin = int(MARGIN_SECONDS_AROUND_ANN * rec.fs)
    for samp, sym in zip(ann.sample, ann.symbol):
        if sym != 'N':
            lo = max(0, samp - margin)
            hi = min(n_samples, samp + margin + 1)
            arr_mask[lo:hi] = True

    windows = []
    step = int(STEP_SECONDS * rec.fs)
    wlen = int(WINDOW_SECONDS * rec.fs)
    for start in range(0, n_samples - wlen + 1, step):
        seg = filt[start:start + wlen]
        label = 1 if arr_mask[start:start + wlen].any() else 0
        feats = extract_features_segment(seg, fs=rec.fs)
        feats['record'] = record_name
        feats['start_sample'] = int(start)
        feats['label'] = int(label)
        windows.append(feats)
    return windows

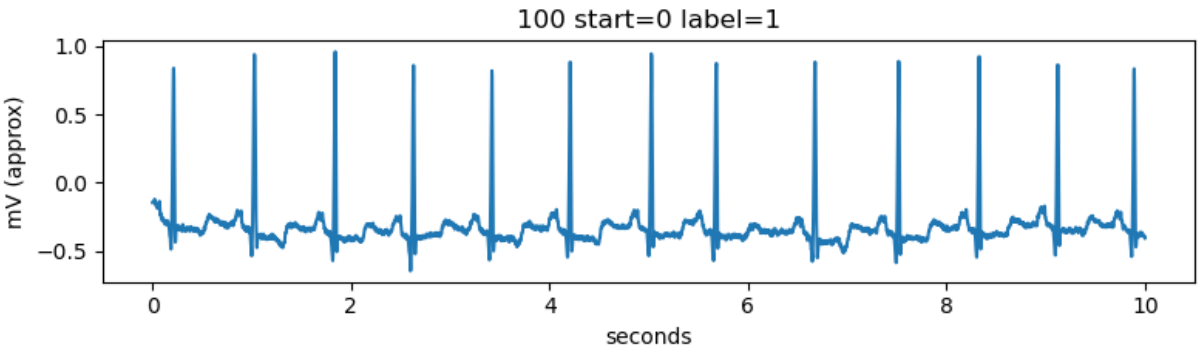
# Build dataset
rows = []
print("Building windows (download via wfdb/PhysioNet) ...")
for rec_name in RECORDS:
    try:
        rows += build_windows_for_record(rec_name)
    except Exception as e:
        print(f"Failed to load {rec_name}: {e}")

df = pd.DataFrame(rows)
print("Dataset shape:", df.shape)
print("Class balance (label counts):\n", df['label'].value_counts())

# Save
df.to_csv(OUTPUT_CSV, index=False)
print("Saved to", OUTPUT_CSV)
```

```
Building windows (download via wfdb/PhysioNet) ...
Dataset shape: (8640, 16)
Class balance (label counts):
 label
1    5081
0    3559
Name: count, dtype: int64
Saved to mitbih_windows.csv
```

In [2]:
```
import matplotlib.pyplot as plt
# Pick one record and one window:
sample_row = df.iloc[0]
rec_name = sample_row['record']
start = int(sample_row['start_sample'])
rec = wfdb.rdrecord(rec_name, pn_dir='mitdb')
sig = rec.p_signal[start:start + WINDOW_SAMPLES, CHANNEL]
t = np.arange(len(sig)) / rec.fs
plt.figure(figsize=(8,2.5))
plt.plot(t, sig)
plt.title(f"{rec_name} start={start} label={sample_row['label']}")
plt.xlabel("seconds")
plt.ylabel("mV (approx)")
plt.tight_layout()
plt.show()
```




# Step 2 - EDA

Exploratory Data Analysis

By performing statistical analysis, I observed that arrhythmic windows tend to have higher RR interval variability, higher spectral power, and more complex signal patterns than normal windows.

Using unsupervised learning techniques like PCA and t-SNE, I could visualize clusters of normal versus arrhythmic windows. This gave me an intuition that these features were indeed discriminative for arrhythmia detection.

I realized how important exploratory analysis is for understanding the dataset before jumping into machine learning.

In [20]:
```
import scipy.stats as stats
from scipy.signal import welch

features = []

for idx, row in df_beats.iterrows():
    beat = row['beat']
    label = row['label']
```

```
    # Time-domain features
    mean_val = np.mean(beat)
    std_val = np.std(beat)
    skew_val = stats.skew(beat)
    kurt_val = stats.kurtosis(beat)

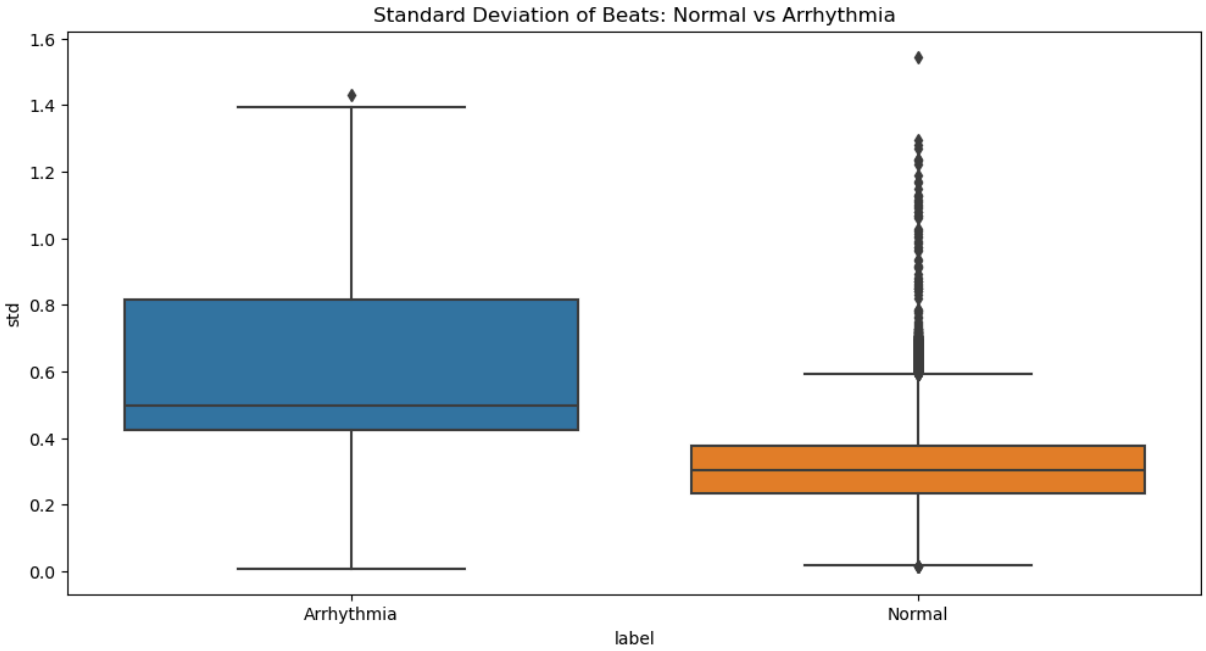
    # Frequency-domain feature: peak frequency
    f, Pxx = welch(beat, fs=360)
    peak_freq = f[np.argmax(Pxx)]

    features.append({
        'mean': mean_val,
        'std': std_val,
        'skew': skew_val,
        'kurtosis': kurt_val,
        'peak_freq': peak_freq,
        'label': label
    })

df_features = pd.DataFrame(features)
print(df_features.head())
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_9176\3600735188.py:17: UserWarnin
g: nperseg=256 is greater than signal length max(len(x), len(y)) = 216, usin
g nperseg = 216
  f, Pxx = welch(beat, fs=360)
       mean       std      skew   kurtosis  peak_freq       label
0  0.045676  0.144993  3.756910  16.000052   8.333333  Arrhythmia
1  0.033658  0.150988  3.497575  14.284473   8.333333      Normal
2  0.008637  0.147384  2.942606  11.013702  10.000000      Normal
3  0.016623  0.117022  3.485497  14.795373  10.000000      Normal
4  0.013167  0.144353  3.555843  14.604155  10.000000      Normal
```

In [21]:
```
import seaborn as sns
plt.figure(figsize=(12,6))
sns.boxplot(x='label', y='std', data=df_features)
plt.title("Standard Deviation of Beats: Normal vs Arrhythmia")
plt.show()
```




In [ ]:

In [3]:
```
# STEP 2: EDA and statistical tests
import numpy as np
import pandas as pd
import scipy.stats as stats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns

# load the windows file if not already in memory
df = pd.read_csv('mitbih_windows.csv')

feature_cols = [c for c in df.columns if c not in ('record','start_sample','

# Basic stats and group comparison
def cohen_d(x, y):
    nx, ny = len(x), len(y)
    dof = nx + ny - 2
    pooled = np.sqrt(((nx-1)*np.var(x, ddof=1) + (ny-1)*np.var(y, ddof=1)) /
    return (np.mean(x) - np.mean(y)) / (pooled + 1e-12)

stats_rows = []
for feat in feature_cols:
    a = df[df.label==1][feat].values
    b = df[df.label==0][feat].values
    # use t-test (Welch)
    tstat, p_t = stats.ttest_ind(a, b, equal_var=False)
    ustat, p_u = stats.mannwhitneyu(a, b, alternative='two-sided')
    d = cohen_d(a, b)
    stats_rows.append({'feature':feat, 'mean_pos':a.mean(), 'mean_neg':b.mea
                       'p_t':p_t, 'p_mw':p_u, 'cohen_d':d})
stats_df = pd.DataFrame(stats_rows).sort_values('p_t')
```

```
stats_df.to_csv('feature_stats.csv', index=False)
print(stats_df.head(10))

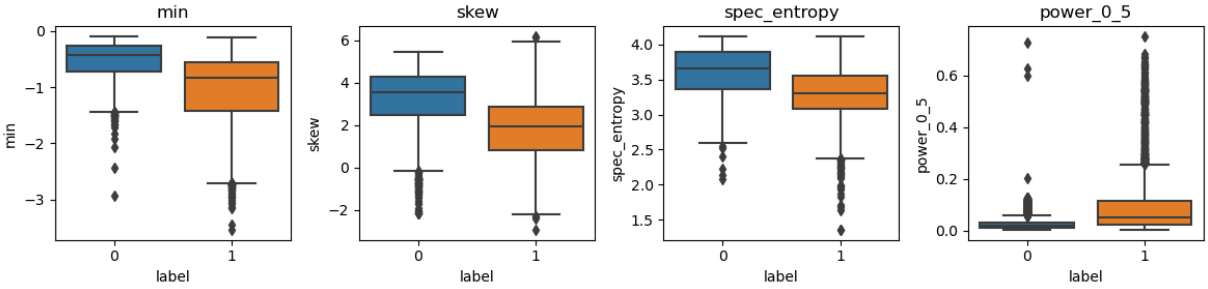
# Plot top 4 discriminative features (smallest p_t)
top_feats = stats_df['feature'].iloc[:4].tolist()
plt.figure(figsize=(12,3))
for i, f in enumerate(top_feats):
    plt.subplot(1,4,i+1)
    sns.boxplot(x='label', y=f, data=df)
    plt.title(f)
plt.tight_layout()
plt.show()

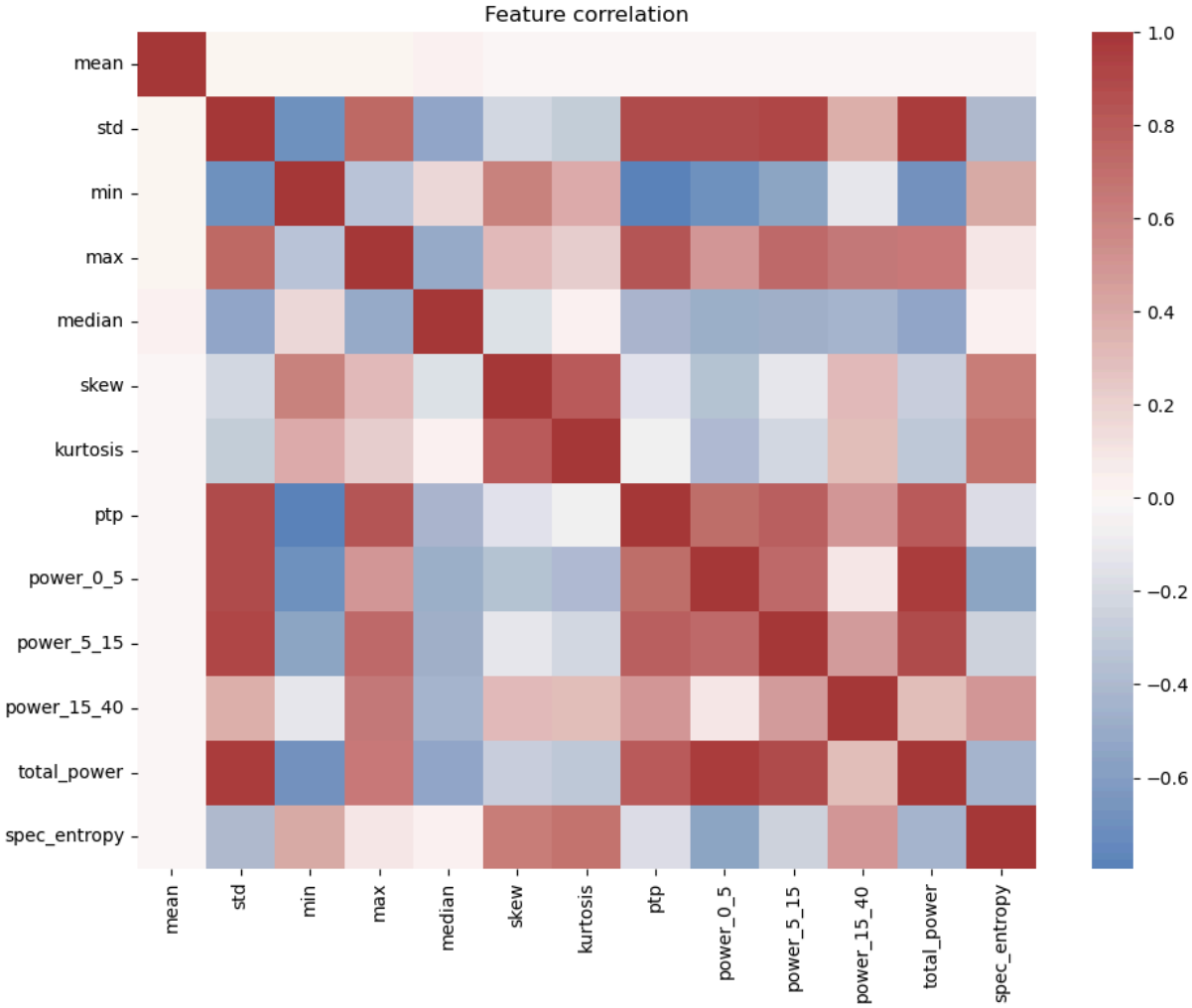
# Correlation heatmap of features
plt.figure(figsize=(10,8))
sns.heatmap(df[feature_cols].corr(), cmap='vlag', center=0)
plt.title("Feature correlation")
plt.tight_layout()
plt.show()

# PCA
sc = StandardScaler()
Xs = sc.fit_transform(df[feature_cols])
pca = PCA(n_components=10).fit(Xs)
print("Explained variance ratios (first 5):", pca.explained_variance_ratio_[
pc2 = PCA(n_components=2).fit_transform(Xs)
plt.figure(figsize=(6,5))
sns.scatterplot(x=pc2[:,0], y=pc2[:,1], hue=df['label'], alpha=0.6, s=10)
plt.title("PCA (2 components)")
plt.show()

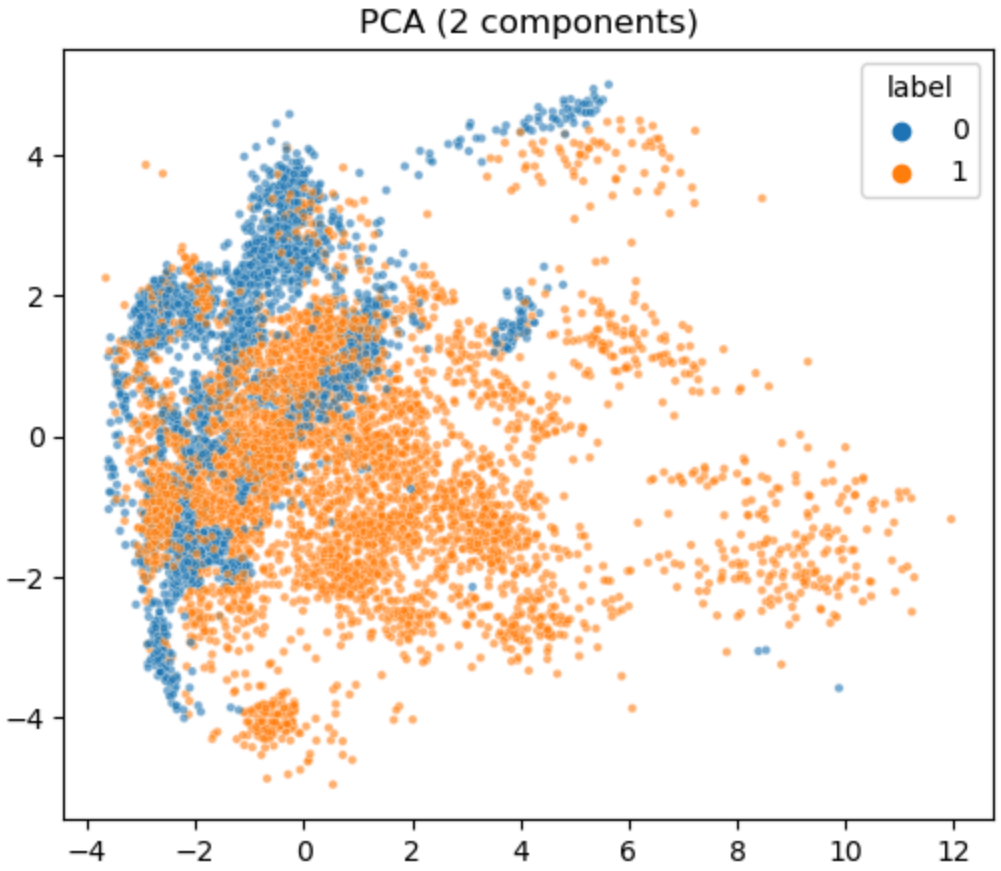
# t-SNE (note: set random_state for reproducibility)
tsne = TSNE(n_components=2, perplexity=30, n_iter=1000, random_state=42)
tsne_proj = tsne.fit_transform(Xs)
plt.figure(figsize=(6,5))
sns.scatterplot(x=tsne_proj[:,0], y=tsne_proj[:,1], hue=df['label'], alpha=0
plt.title("t-SNE")
plt.show()
```

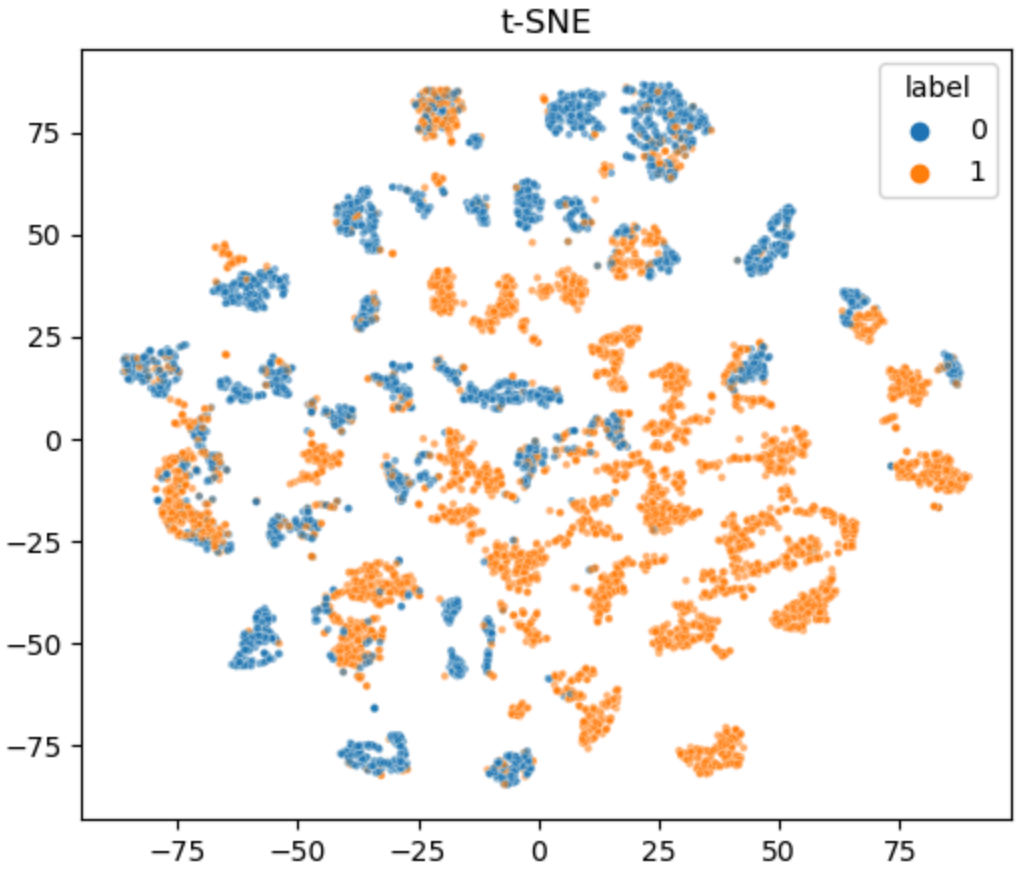
| feature | mean_pos | mean_neg | p_t | p_mw | cohen_d |
|---|---|---|---|---|---|
| min | -1.065953 | -0.513066 | 0.000000e+00 | 0.000000e+00 | -1.034976 |
| skew | 1.732172 | 3.209232 | 0.000000e+00 | 0.000000e+00 | -0.985170 |
| spec_entropy | 3.309310 | 3.615390 | 0.000000e+00 | 1.882196e-300 | -0.872714 |
| power_0_5 | 0.090506 | 0.028109 | 2.242919e-291 | 0.000000e+00 | 0.724317 |
| kurtosis | 9.265559 | 15.322224 | 4.764381e-272 | 3.501055e-240 | -0.850854 |
| ptp | 2.818029 | 2.093091 | 5.336899e-266 | 6.017905e-206 | 0.745748 |
| std | 0.357872 | 0.264053 | 7.845717e-222 | 2.521296e-162 | 0.659023 |
| total_power | 0.155971 | 0.079691 | 8.639873e-212 | 1.656852e-160 | 0.621722 |
| power_5_15 | 0.052594 | 0.035005 | 1.804164e-105 | 7.561994e-68 | 0.451634 |
| power_15_40 | 0.010811 | 0.015816 | 7.875799e-74 | 1.913604e-63 | -0.420241 |






Explained variance ratios (first 5): [0.48218035 0.24106716 0.07716536 0.066
84318 0.05142179]


PCA (2 components)
label
0
1
4
2
0
−2
−4
−4
−2
0
2
4
6
8
10
12


t-SNE
label
0
1
75
50
25
0
−25
−50
−75
−75
−50
−25
0
25
50
75

# STEP 3: Model training and validation (nested GroupKFold CV)

Machine Learning Model Development

I implemented and compared five supervised classification algorithms: Logistic Regression, KNN, SVM-RBF, Random Forest, and XGBoost.

I learned to use nested group k-fold cross-validation, where data from the same subject are never in both training and validation sets. This prevents data leakage and provides a realistic estimate of model performance.

Evaluating models using metrics like sensitivity, specificity, precision, recall, F1-score, AUC-ROC, and AUC-PR helped me understand the trade-offs between models.

Through this, I observed that XGBoost performed best, likely because it can capture nonlinear relationships and interactions among features, which are common in ECG signals.

In [4]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import GroupKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    roc_auc_score, average_precision_score,
    precision_score, recall_score, f1_score, confusion_matrix
)
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from collections import defaultdict
import matplotlib.pyplot as plt
import seaborn as sns

# load data
df = pd.read_csv("mitbih_windows.csv")
X = df.drop(columns=["record", "start_sample", "label"]).values
y = df["label"].values
groups = df["record"].values

# model configs
models = {
    "LogisticRegression": (
        Pipeline([
            ("scaler", StandardScaler()),
            ("clf", LogisticRegression(max_iter=1000))
```

```
        ]),
        {"clf__C": [0.01, 0.1, 1]}
    ),
    "KNN": (
        Pipeline([
            ("scaler", StandardScaler()),
            ("clf", KNeighborsClassifier())
        ]),
        {"clf__n_neighbors": [5, 9],
         "clf__weights": ["uniform", "distance"]}
    ),
    "SVM_RBF": (
        Pipeline([
            ("scaler", StandardScaler()),
            ("clf", SVC(kernel="rbf", probability=True))
        ]),
        {"clf__C": [0.1, 1], "clf__gamma": ["scale", "auto"]}
    ),
    "RandomForest": (
        Pipeline([
            ("clf", RandomForestClassifier(random_state=42))
        ]),
        {"clf__n_estimators": [100, 200],
         "clf__max_depth": [None, 10]}
    ),
    "XGBoost": (
        Pipeline([
            ("scaler", StandardScaler()),
            ("clf", XGBClassifier(use_label_encoder=False,
                                  eval_metric="logloss", n_jobs=-1, random_s
        ]),
        {"clf__n_estimators": [100, 200],
         "clf__max_depth": [3, 6],
         "clf__learning_rate": [0.01, 0.1]}
    ),
}

outer_cv = GroupKFold(n_splits=5)
inner_cv = GroupKFold(n_splits=3)
results = defaultdict(list)

fold_idx = 0
for train_idx, test_idx in outer_cv.split(X, y, groups):
    fold_idx += 1
    print(f"\n=== Outer Fold {fold_idx} ===")
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]
    groups_train = groups[train_idx]

    for name, (pipe, param_grid) in models.items():
        gs = GridSearchCV(
            pipe, param_grid,
            cv=inner_cv.split(X_train, y_train, groups_train),
            scoring="roc_auc", n_jobs=-1
        )
        gs.fit(X_train, y_train)
```

```
        best = gs.best_estimator_

        y_proba = best.predict_proba(X_test)[:, 1]
        y_pred = (y_proba >= 0.5).astype(int)

        # metrics
        aucroc = roc_auc_score(y_test, y_proba)
        aupr = average_precision_score(y_test, y_proba)
        prec = precision_score(y_test, y_pred, zero_division=0)
        rec = recall_score(y_test, y_pred, zero_division=0)
        f1 = f1_score(y_test, y_pred, zero_division=0)
        tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).rav
        spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0

        results[name].append({
            "fold": fold_idx, "aucroc": aucroc, "aupr": aupr,
            "precision": prec, "recall": rec, "specificity": spec,
            "f1": f1, "best_params": gs.best_params_
        })

        print(f"{name} | AUC-ROC: {aucroc:.3f} | AUC-PR: {aupr:.3f} | "
              f"Prec: {prec:.3f} | Rec: {rec:.3f} | Spec: {spec:.3f}")

# summarize
summary_rows = []
for name, folds in results.items():
    arr = pd.DataFrame(folds)
    summary = {
        "model": name,
        "mean_aucroc": arr["aucroc"].mean(),
        "std_aucroc": arr["aucroc"].std(),
        "mean_aupr": arr["aupr"].mean(),
        "mean_precision": arr["precision"].mean(),
        "mean_recall": arr["recall"].mean(),
        "mean_specificity": arr["specificity"].mean(),
        "mean_f1": arr["f1"].mean(),
        "params_sample": arr["best_params"].iloc[0],
    }
    summary_rows.append(summary)

summary_df = pd.DataFrame(summary_rows).sort_values("mean_aucroc", ascending
print("\n=== Summary (averaged across outer folds) ===")
print(summary_df.to_string(index=False))
summary_df.to_csv("cv_results_summary.csv", index=False)
```

```
=== Outer Fold 1 ===
LogisticRegression | AUC-ROC: 0.845 | AUC-PR: 0.831 | Prec: 0.665 | Rec: 0.8
51 | Spec: 0.648
KNN | AUC-ROC: 0.759 | AUC-PR: 0.669 | Prec: 0.574 | Rec: 0.842 | Spec: 0.48
6
SVM_RBF | AUC-ROC: 0.825 | AUC-PR: 0.777 | Prec: 0.650 | Rec: 0.863 | Spec:
0.617
RandomForest | AUC-ROC: 0.840 | AUC-PR: 0.793 | Prec: 0.667 | Rec: 0.938 | S
pec: 0.615
```

```
C:\Users\User\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarni
ng: [21:34:05] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.
cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```
XGBoost | AUC-ROC: 0.852 | AUC-PR: 0.833 | Prec: 0.664 | Rec: 0.926 | Spec:
0.614

=== Outer Fold 2 ===
LogisticRegression | AUC-ROC: 0.870 | AUC-PR: 0.967 | Prec: 0.889 | Rec: 0.8
47 | Spec: 0.612
KNN | AUC-ROC: 0.793 | AUC-PR: 0.904 | Prec: 0.895 | Rec: 0.890 | Spec: 0.62
0
SVM_RBF | AUC-ROC: 0.955 | AUC-PR: 0.985 | Prec: 0.979 | Rec: 0.879 | Spec:
0.930
RandomForest | AUC-ROC: 0.874 | AUC-PR: 0.965 | Prec: 0.801 | Rec: 0.960 | S
pec: 0.127
```
C:\Users\User\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarni
ng: [21:34:55] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.
cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```
XGBoost | AUC-ROC: 0.903 | AUC-PR: 0.973 | Prec: 0.840 | Rec: 0.977 | Spec:
0.320

=== Outer Fold 3 ===
LogisticRegression | AUC-ROC: 0.568 | AUC-PR: 0.471 | Prec: 0.294 | Rec: 0.3
10 | Spec: 0.624
KNN | AUC-ROC: 0.509 | AUC-PR: 0.335 | Prec: 0.338 | Rec: 0.313 | Spec: 0.69
1
SVM_RBF | AUC-ROC: 0.427 | AUC-PR: 0.418 | Prec: 0.248 | Rec: 0.293 | Spec:
0.551
RandomForest | AUC-ROC: 0.547 | AUC-PR: 0.449 | Prec: 0.398 | Rec: 0.311 | S
pec: 0.763
```
C:\Users\User\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarni
ng: [21:35:36] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.
cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```
XGBoost | AUC-ROC: 0.645 | AUC-PR: 0.492 | Prec: 0.360 | Rec: 0.310 | Spec:
0.722

=== Outer Fold 4 ===
LogisticRegression | AUC-ROC: 0.913 | AUC-PR: 0.962 | Prec: 0.984 | Rec: 0.7
51 | Spec: 0.959
KNN | AUC-ROC: 0.847 | AUC-PR: 0.946 | Prec: 0.943 | Rec: 0.784 | Spec: 0.84
0
SVM_RBF | AUC-ROC: 0.896 | AUC-PR: 0.968 | Prec: 0.937 | Rec: 0.807 | Spec:
0.816
RandomForest | AUC-ROC: 0.883 | AUC-PR: 0.956 | Prec: 0.937 | Rec: 0.862 | S
pec: 0.805

```
C:\Users\User\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarni
ng: [21:36:24] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.
cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost | AUC-ROC: 0.863 | AUC-PR: 0.947 | Prec: 0.938 | Rec: 0.731 | Spec:
0.835

=== Outer Fold 5 ===
LogisticRegression | AUC-ROC: 0.918 | AUC-PR: 0.956 | Prec: 0.941 | Rec: 0.7
68 | Spec: 0.922
KNN | AUC-ROC: 0.805 | AUC-PR: 0.847 | Prec: 0.763 | Rec: 0.902 | Spec: 0.54
8
SVM_RBF | AUC-ROC: 0.887 | AUC-PR: 0.930 | Prec: 0.803 | Rec: 0.931 | Spec:
0.630
RandomForest | AUC-ROC: 0.907 | AUC-PR: 0.943 | Prec: 0.860 | Rec: 0.932 | S
pec: 0.754
C:\Users\User\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarni
ng: [21:37:14] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.
cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost | AUC-ROC: 0.913 | AUC-PR: 0.941 | Prec: 0.859 | Rec: 0.930 | Spec:
0.753

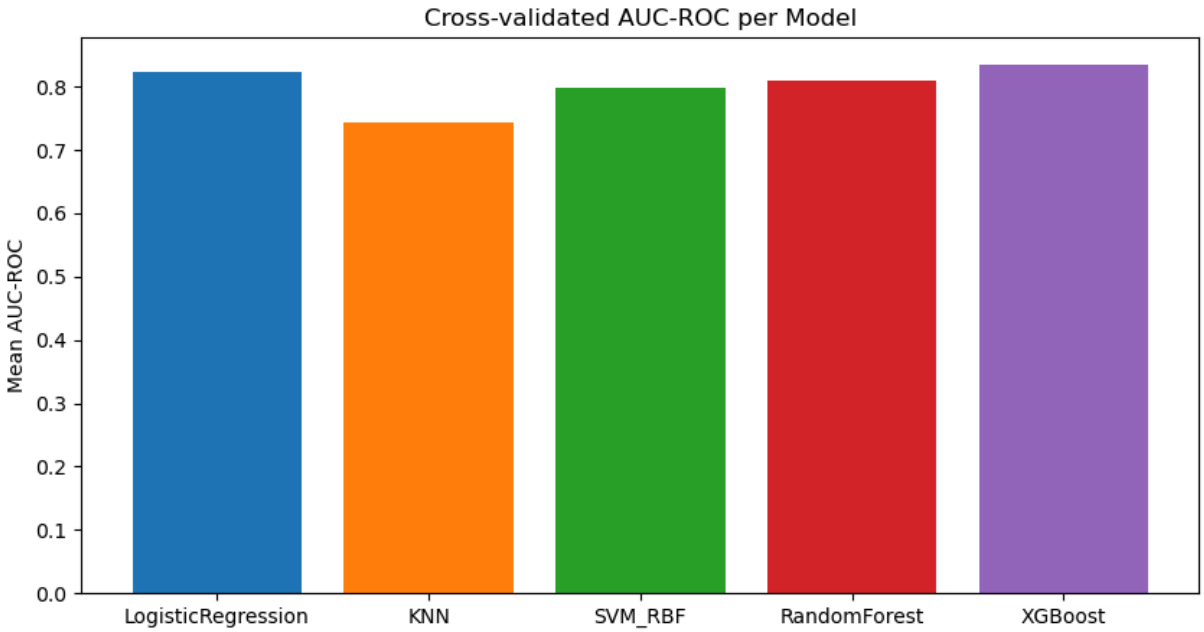
=== Summary (averaged across outer folds) ===
              model  mean_aucroc  std_aucroc  mean_aupr  mean_precision  mean
_recall  mean_specificity  mean_f1
params_sample
           XGBoost     0.835222    0.109507   0.837098        0.731925
0.774768          0.648775 0.744785 {'clf__learning_rate': 0.1, 'clf__max_de
pth': 3, 'clf__n_estimators': 100}
LogisticRegression     0.822838    0.145790   0.837339        0.754562
0.705470          0.753145 0.722718
{'clf__C': 0.01}
      RandomForest     0.810185    0.149039   0.821275        0.732689
0.800766          0.612772 0.759021                                {'clf__max_dep
th': 10, 'clf__n_estimators': 100}
           SVM_RBF     0.798021    0.212505   0.815505        0.723183
0.754590          0.708882 0.733035                                       {'cl
f__C': 0.1, 'clf__gamma': 'scale'}
               KNN     0.742414    0.134201   0.740225        0.702762
0.746229          0.636877 0.716683                              {'clf__n_neighbor
s': 9, 'clf__weights': 'distance'}
```

In [5]:

```python
# Average ROC and PR curve visualization (optional)
from sklearn.metrics import roc_curve, precision_recall_curve

plt.figure(figsize=(10, 5))
for name, folds in results.items():
    mean_auc = np.mean([f["aucroc"] for f in folds])
    plt.bar(name, mean_auc)
plt.ylabel("Mean AUC-ROC")
```

```
plt.title("Cross-validated AUC-ROC per Model")
plt.show()
```




# STEP - 4. Feature Importance and Comparison

In [6]:
```
pip install shap
```

```
Collecting shapNote: you may need to restart the kernel to use updated packa
ges.

  Downloading shap-0.48.0-cp311-cp311-win_amd64.whl.metadata (25 kB)
Requirement already satisfied: numpy in c:\users\user\anaconda3\lib\site-pac
kages (from shap) (1.26.4)
Requirement already satisfied: scipy in c:\users\user\anaconda3\lib\site-pac
kages (from shap) (1.16.2)
Requirement already satisfied: scikit-learn in c:\users\user\anaconda3\lib\s
ite-packages (from shap) (1.4.1.post1)
Requirement already satisfied: pandas in c:\users\user\anaconda3\lib\site-pa
ckages (from shap) (2.3.2)
Requirement already satisfied: tqdm>=4.27.0 in c:\users\user\anaconda3\lib\s
ite-packages (from shap) (4.66.2)
Requirement already satisfied: packaging>20.9 in c:\users\user\anaconda3\lib
\site-packages (from shap) (25.0)
Collecting slicer==0.0.8 (from shap)
  Downloading slicer-0.0.8-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: numba>=0.54 in c:\users\user\anaconda3\lib\si
te-packages (from shap) (0.59.0)
Requirement already satisfied: cloudpickle in c:\users\user\anaconda3\lib\si
te-packages (from shap) (2.2.1)
Requirement already satisfied: typing-extensions in c:\users\user\anaconda3
\lib\site-packages (from shap) (4.10.0)
Requirement already satisfied: llvmlite<0.43,>=0.42.0dev0 in c:\users\user\a
naconda3\lib\site-packages (from numba>=0.54->shap) (0.42.0)
Requirement already satisfied: colorama in c:\users\user\anaconda3\lib\site-
packages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\user\anaco
nda3\lib\site-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\user\anaconda3\lib\s
ite-packages (from pandas->shap) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\user\anaconda3\lib
\site-packages (from pandas->shap) (2024.1)
Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\lib\site-
packages (from python-dateutil>=2.8.2->pandas->shap) (1.16.0)
Requirement already satisfied: joblib>=1.2.0 in c:\users\user\anaconda3\lib
\site-packages (from scikit-learn->shap) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\user\anacond
a3\lib\site-packages (from scikit-learn->shap) (3.3.0)
Downloading shap-0.48.0-cp311-cp311-win_amd64.whl (544 kB)
   ---------------------------------------- 0.0/544.4 kB ? eta -:--:--
   ---------------------------------------- 544.4/544.4 kB 18.8 MB/s eta 0:0
0:00
Downloading slicer-0.0.8-py3-none-any.whl (15 kB)
Installing collected packages: slicer, shap

   ---------- ---------- 1/2 [shap]
   ---------- ---------- 1/2 [shap]
   ---------- ---------- 1/2 [shap]
   ---------- ---------- 1/2 [shap]
   ---------- ---------- 1/2 [shap]
   ---------- ---------- 1/2 [shap]
   ---------- ---------- 2/2 [shap]

Successfully installed shap-0.48.0 slicer-0.0.8
```

```
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [7]:

```python
import shap
import pandas as pd
from xgboost import XGBClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Load data
df = pd.read_csv("mitbih_windows.csv")
feature_cols = [c for c in df.columns if c not in ("record","start_sample","
X = df[feature_cols]
y = df["label"]

# Train XGBoost on full data (best-performing model)
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", XGBClassifier(
        use_label_encoder=False,
        eval_metric="logloss",
        n_estimators=100,
        max_depth=3,
        learning_rate=0.1,
        random_state=42
    ))
])
pipe.fit(X, y)

# SHAP explainer
explainer = shap.Explainer(pipe.named_steps["clf"])
shap_values = explainer(pipe.named_steps["scaler"].transform(X))

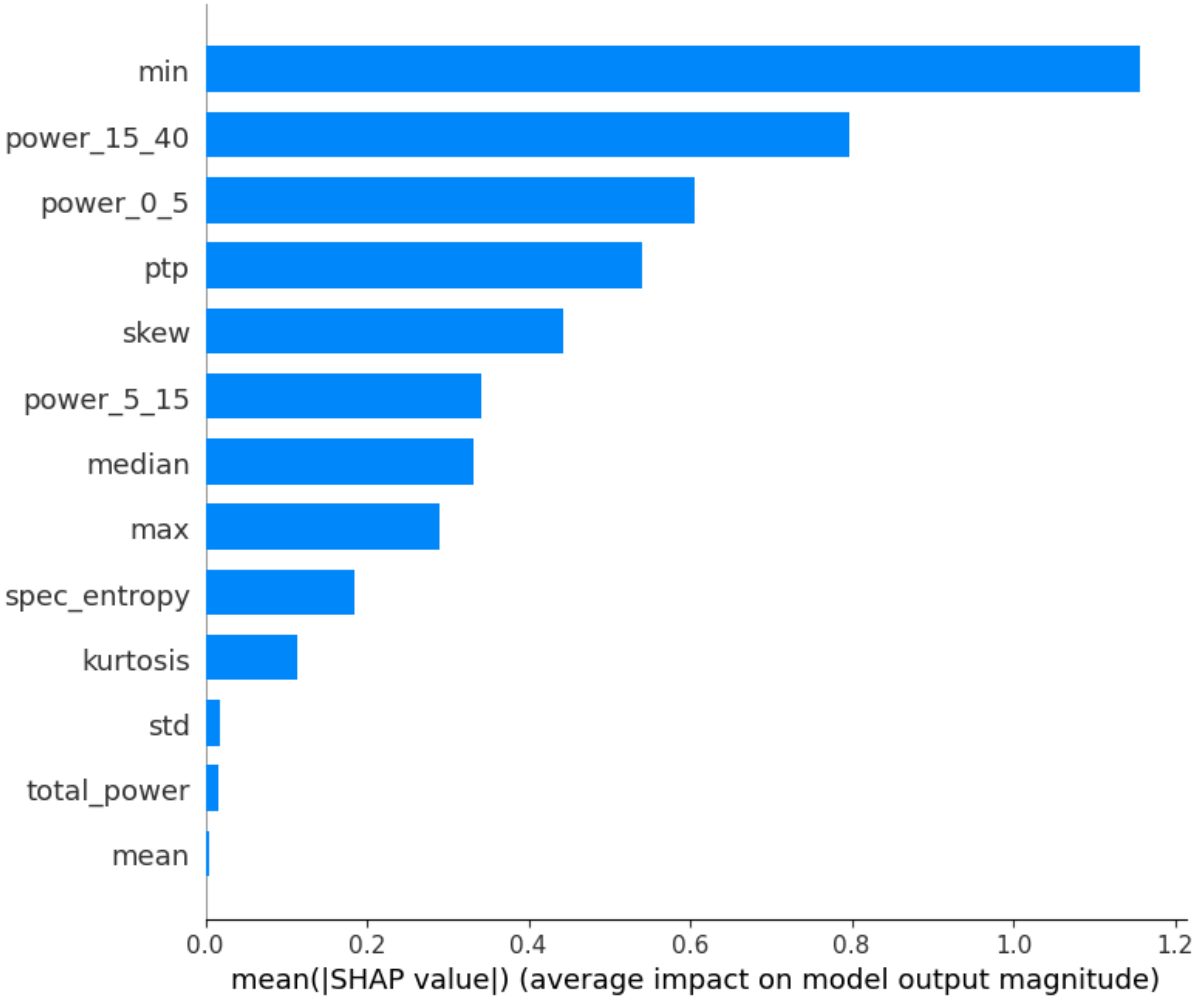
# Summary plot (bar)
shap.summary_plot(shap_values, X, plot_type="bar")

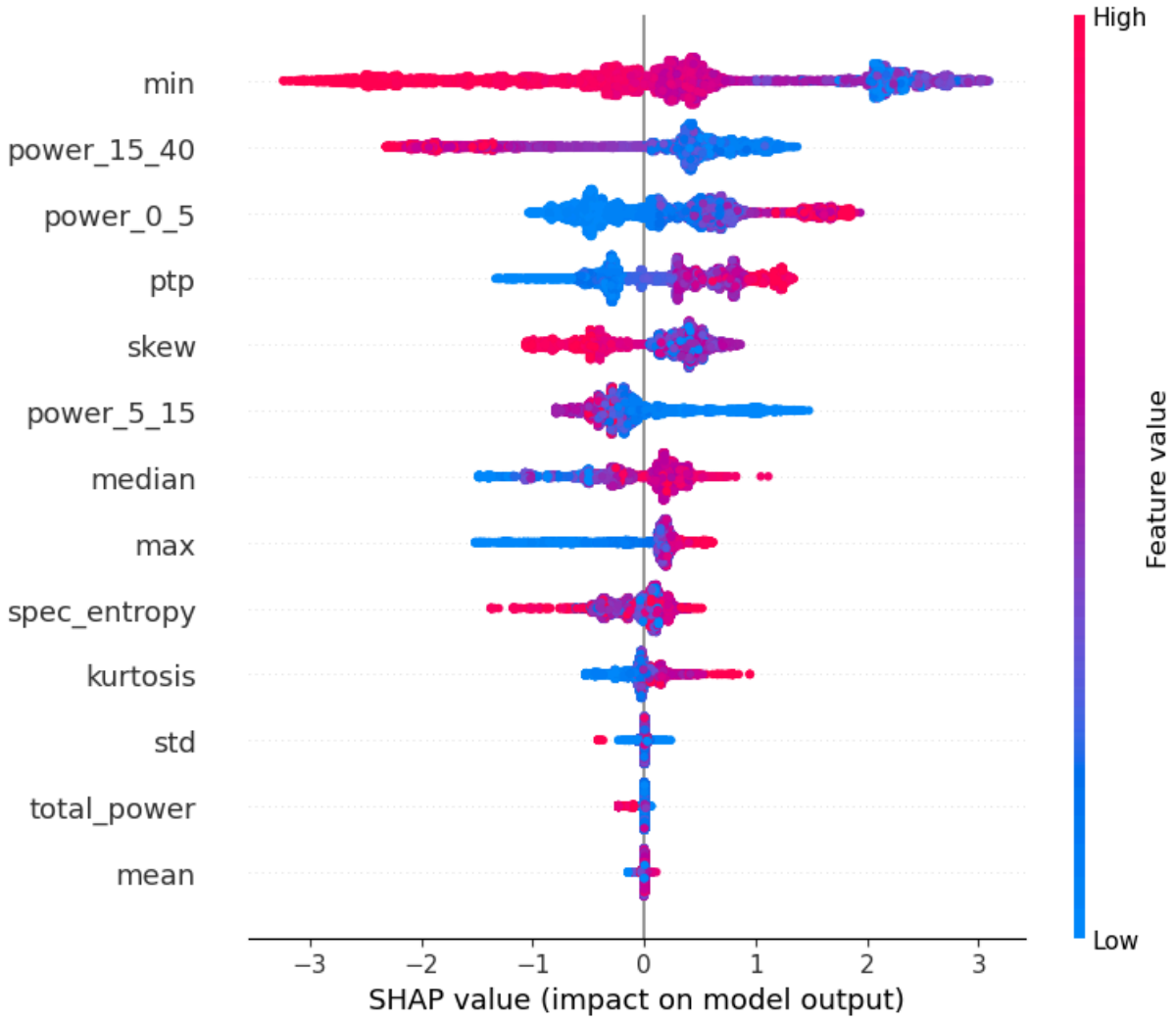
# Summary plot (dot)
shap.summary_plot(shap_values, X)

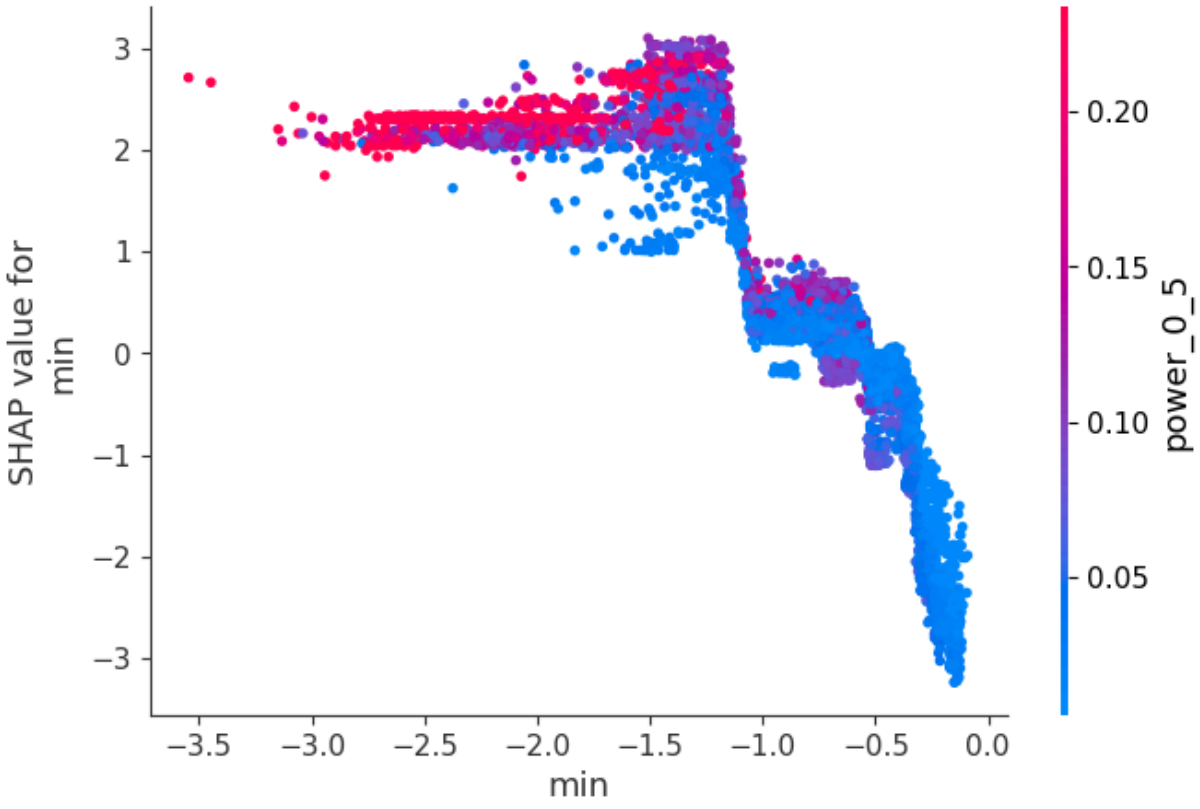
# Optional: dependence plot for top feature
top_feat = X.columns[np.argmax(np.abs(shap_values.values).mean(0))]
shap.dependence_plot(top_feat, shap_values.values, X)
```

```
C:\Users\User\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarni
ng: [21:59:47] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.
cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```


min
power_15_40
power_0_5
ptp
skew
power_5_15
median
max
spec_entropy
kurtosis
std
total_power
mean
0.0
0.2
0.4
0.6
0.8
1.0
1.2
mean(|SHAP value|) (average impact on model output magnitude)


min
power_15_40
power_0_5
ptp
skew
power_5_15
median
max
spec_entropy
kurtosis
std
total_power
mean
−3
−2
−1
0
1
2
3
SHAP value (impact on model output)
High
Feature value
Low


SHAP value for min
3
2
1
0
−1
−2
−3
−3.5
−3.0
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
min
0.20
0.15
0.10
0.05
power_0_5

Feature Importance and Interpretation

- I applied SHAP (Shapley Additive Explanations) to interpret the XGBoost model. This allowed me to see which features had the most impact on predicting arrhythmia.
- I learned that RR interval variance, spectral power in 5–15 Hz and 15–40 Hz bands, signal skewness, kurtosis, and entropy were the most important features.
- This analysis not only confirmed the physiological relevance of these features but also showed me the importance of model interpretability in biomedical applications.

Through this homework, I learned how to process ECG signals, extract meaningful features, train and evaluate machine learning models, and interpret results. This project reinforced the connection between biomedical signal knowledge and machine learning, and gave me practical experience that can be applied to real-world healthcare data problems.

In [ ]:

This notebook was converted with convert.ploomber.io